

```

x=6869
y=7873
z=x*y

%time
a = z**100

CPU times: user 1 µs, sys: 2 µs, total: 3 µs
Wall time: 6.2 µs

%time
b = x**100 * y**100

CPU times: user 2 µs, sys: 0 ns, total: 2 µs
Wall time: 3.81 µs

a==b

True

```

Figure 4: Execution time of a Python code

Now apply lines 4 to 10 on matrix B. Lines 4 to 7 work as before. However, the determinant of B is 0 and hence it is called a singular matrix. Further, line 8 gives you an error because inverse exists only for non-singular matrices whose determinant is non-zero. Now, apply the same operations on all the eight matrices we have introduced in the previous article in this series. On seeing the outputs, you will get the additional insight that matrix operations like determinant and inverse are applicable only to square matrices.

Square matrices are matrices with an equal number of rows and columns. Though we tried to understand the working of these operations based on the above examples, I haven't explained anything about the theory behind them. So, it would be a good idea to learn more about matrix operations like transpose, inverse, determinant, etc, if you have forgotten them. You could also learn about the different types of matrices like identity matrix, diagonal matrix, triangular matrix, symmetric matrix, skew symmetric matrix, etc.

Wikipedia articles on these terms are a good starting point.

Now let us learn about some matrix operations that are even more complex, called matrix decomposition (also known as matrix factorization). First, let us recall integer factorization to understand the need for matrix factorization. The number 15 is the product of 3 and 5. But why do we need factorization? Consider the semi-prime (an integer which is the product of exactly two prime numbers) 54079637 whose factors are 6869 and 7873. I think the code and the output shown in Figure 4 give an answer to our question about the need for factorization. The line of code `%time` in the beginning shows us the time taken to execute a cell in Jupyter Notebook. Elementary number theory tells us that both the variables a and b contain the answer to the mathematical expression 54079637×100 . However, Figure 4 shows us that the answer in variable b is calculated much faster. Further, this reduction in execution time will become better and better as the number and the exponent in the above

calculation increases.

Now let us move on to matrix decomposition. Similar to integer factorization, in matrix decomposition a matrix is written as the product of some other matrices. There are a lot of matrix decomposition techniques, and in almost all of them a matrix is written as the product of matrices that are more sparse than the original matrix. A sparse matrix is a matrix that has a lot of elements with value zero. Thus, after decomposition, we can work with sparse matrices rather than the original dense matrix with a lot of non-zero elements. In this article, we will discuss three decomposition techniques -- LUP decomposition, eigendecomposition, and singular value decomposition (SVD).

But, in order to perform matrix decomposition, we need the services of yet another powerful Python library called SciPy. SciPy has functions for performing operations in linear algebra, integration, differentiation, optimisation, etc. In a sense, SciPy works on top of NumPy. First, let us discuss LUP decomposition. Any square matrix has an LUP decomposition. There is a variant of LUP decomposition called LU decomposition. However, not every square matrix has an LU decomposition. Hence, we discuss LUP decomposition.

In LUP decomposition, a matrix A is written as the product of three matrices L, U, and P. However, these three matrices do have some properties. L is a lower triangular matrix. A square matrix with all the entries above the main diagonal as zero is called a lower triangular matrix. U is an upper triangular matrix. A square matrix with all the entries below the main diagonal as zero is called an upper triangular matrix. P is a permutation matrix. This is a square matrix which has exactly one element with value 1 in each row and each column, and every other element of it has the value 0.

Now consider the code given below to perform LUP decomposition. Again, the line numbers have been